Analysis plan — universality and individuality of infant EEG network constituents

Oct 20, 2026 · @Adi B

Purpose and overview

The goal is to separate, for every node and edge centrality, what is shared by all infants (universality) from what is specific to each infant (individuality). We do this with four comparisons, all using the same weighted Jaccard similarity, and then combine them in one identifiability framework.

Comparison	Recordings compared	What it measures	Status
A. Universality	Infant a vs infant b, same session	How much of the network organisation is shared across infants	Done for ses-1 (redo with the rules below); to do for ses-2
B. Within-session self-similarity	Infant a, first half of epochs vs second half, same session	Moment-to-moment (state) variability; the ceiling for self-similarity	To do
C. Between-session self-similarity	Infant a at ses-1 vs infant a at ses-2	Individual stable signature (trait-like), mixed with development	To do
D. Between-session others	Infant a at ses-1 vs infant b at ses-2	Null reference for C: same time gap, different infant	To do

Individuality is not C alone. It is C minus D: how much more an infant resembles itself three months later than it resembles other infants three months later. Steps 0–8 below specify every computation; nothing should need to be assumed.

Notation and inputs

Every analysis below reads from one array per feature, X_f, of shape (20 infants × 2 sessions × K epochs × n elements), where n = 19 for node features and 171 for edge features.

- **Infants** a, b ∈ {1, …, 20}, always in the same fixed order (sorted by participant ID).
- **Sessions** s, t ∈ {1, 2} (ses-1, ses-2).
- **Epochs** k = 1, …, K: the first K expert-annotated epochs in chronological order, with the same K in both sessions (Step 0).
- **Elements** e: one of the 19 nodes or 171 edges. Keep the existing order unchanged in every array, matrix, table and figure: nodes ordered by region (F, C, T, P, O), and edges ordered by region pair with increasing distance between regions (F–F, F–C, F–T, F–P, F–O, C–C, C–T, C–P, C–O, T–T, T–P, T–O, P–P, P–O, O–O), as in the existing code.
- **Feature value** x_{a,s,k}(e) ⩾ 0: the value of feature f at element e for infant a, session s, epoch k.
- **Recording** R = (a, s): one infant in one session.

Feature. A feature is one centrality measure computed in one network representation (weighted or binary). It gives one value per node (19 values) or per edge (171 values) for every infant, session and epoch. There are 18 feature sets: 4 node centralities (strength/degree, eigenvector, closeness, betweenness) and 5 edge centralities (weight/adjacency, nearest-neighbour edge strength, eigenvector, closeness, betweenness), each in weighted and binary networks.

Frequency bands. For delta, theta, alpha and beta, compute the weighted edge weight W_ij and the binary edge adjacency A_ij (8 more feature sets), with the existing band filters. Binarise each band-filtered network with the same GCC rule as broadband, per infant and per epoch.

Step 0 — Preparation

Before any similarity is computed, ses-2 must be processed exactly like ses-1 and a few bookkeeping items must be fixed.

1. **Epoch count K.** Count the annotated epochs at ses-2 for all 20 infants. If every infant has ⩾ 12 at both sessions, use K = 12 for both sessions. If any infant has fewer than 12, stop and report the counts; do not drop infants or change K silently. K must be even (Step 3 splits it in half).

2. **Features for ses-2.** Compute all 18 feature sets (plus the 8 band-specific edge sets) for ses-2 with the identical pipeline: first K epochs in chronological order, |r| weights, GCC thresholding per infant and per epoch with the same grid (start 1.0, step 0.05, keep W ⩾ τ), distance 1/w.
3. **Closeness (both sessions).** Node closeness: `nx.closeness_centrality(G, distance="distance")` with distance = 1/w for weighted graphs and no distance argument for binary graphs; for a connected graph this equals Eq. (8). Edge closeness: implement Eq. (9) exactly as in the code used for the Communications Physics paper, and check it on a small toy graph against values computed by hand.
4. **Possible twins.** NORB00086 and NORB00087 have identical ages at both sessions (0.2377 y and 0.525 y). Check participants.tsv and the scans.tsv acquisition times. If they are twins or siblings, flag the pair in all outputs, run the sibling analysis at the end of Step 6, and also report whether they were recorded on the same date at each session. Step 8 reruns the main results without the pair.
5. **Threshold bookkeeping.** For every binary network, store τ_c and the edge density. Report the minimum, median and maximum of each, per session; these numbers go into the Methods.
6. **Metadata table**, one row per infant: participant ID, sex, age at ses-1 and ses-2 (days), inter-session interval (days), number of annotated epochs at each session.
7. **Element table**, one row per node or edge: label (e.g., "F3–C3"), region or region pair, hemispheric category, within- or between-region, and for edges the Euclidean distance between the two electrodes. Take coordinates from the dataset's electrodes.tsv; if absent, use the MNE "standard_1020" montage with T3 = T7, T4 = T8, T5 = P7, T6 = P8.
8. **Reproducibility.** Fix one random seed for all bootstrap and permutation steps and record it.

Step 1 — Similarity measures

Two similarity measures are used throughout: an element-level index (one value per node or edge) and a recording-level index (one value per pair of recordings). Both compare two recordings R1 and R2 over a list P of paired epochs.

Element-level weighted Jaccard. For one epoch pair (k1, k2) and element e:

```
J^{(k_1,k_2)}_{R_1R_2}(e) = \frac{\min\{x_{R_1,k_1}(e),\, x_{R_2,k_2}(e)\}}
{\max\{x_{R_1,k_1}(e),\, x_{R_2,k_2}(e)\}}
```

Then average over the epoch pairs in P: J_{R1R2}(e) = mean over P of the values above.

- **0/0 rule (changed from the current code).** When both values are zero, the ratio is undefined: set it to NaN and leave it out of the average (nanmean). This matches the classical Jaccard index, which ignores joint absences. If all pairs are NaN for an element, J is NaN for that element; report how often this happens per element. The current rule (0/0 = 0) is kept only as a sensitivity check (Step 8).
- **Redo ses-1.** Recompute all existing ses-1 results with this rule so that every result in the paper uses the same definition.

Recording-level weighted Jaccard. For one epoch pair, sum over all elements of the feature (19 or 171), then average over P:

```
S^{(k_1,k_2)}_{R_1R_2} = \frac{\sum_{e} \min\{x_{R_1,k_1}(e),\, x_{R_2,k_2}(e)\}}{\sum_{e} \max\{x_{R_1,k_1}(e),\, x_{R_2,k_2}(e)\}}, \qquad S_{R_1R_2} = \frac{1}{|P|} \sum_{(k_1,k_2) \in P} S^{(k_1,k_2)}_{R_1R_2}
```

Elements that are zero in both recordings add 0 to both sums, so no special rule is needed. For binary adjacency, S is exactly the classical Jaccard index of the two edge sets.

Epoch pairing P.

Comparison	P	Number of pairs
A, C, D (two different recordings)	(k, k) for k = 1, …, K	K
B (two halves of one recording)	(k, k + K/2) for k = 1, …, K/2	K/2

Pairing by index is arbitrary, because resting EEG is not time-locked across recordings; Step 8 repeats the main results averaging over all K × K pairs.

Reference implementation (x1, x2: arrays of shape (number of pairs, number of elements), rows already paired):

```
import numpy as np

def element_J(x1, x2):
    mn, mx = np.minimum(x1, x2), np.maximum(x1, x2)
    j = np.full(mn.shape, np.nan)
    ok = mx > 0
    j[ok] = mn[ok] / mx[ok]
    return np.nanmean(j, axis=0)          # one value per element

def recording_S(x1, x2):
    return np.mean(np.minimum(x1, x2).sum(axis=1) /
                   np.maximum(x1, x2).sum(axis=1))   # one value
```

Step 2 — Universality: between infants within a session (Comparison A)

For every feature and for each session separately (s = 1 and s = 2), compare every pair of different infants in that session, with epoch pairing (k, k).

1. **Element level.** Compute J_{(a,s),(b,s)}(e) for all 190 pairs a < b. This gives a 190 × n matrix per feature and session (the existing heatmap). Universality of element e: U_s(e) = mean over the 190 pairs.
2. **Recording level.** Compute S_{(a,s),(b,s)} for all 190 pairs and store it as a symmetric 20 × 20 matrix with an empty diagonal. Overall universality U_s = mean of the 190 values.
3. **Per-infant universality.** u_s(a) = mean over the 19 other infants b of S_{(a,s),(b,s)}. These 20 values are used for infant-level statistics (Step 7).
4. **Regions.** For each region (nodes) or region pair (edges) g, take the mean of J over the elements in g for each pair, giving a 190 × G matrix; U_s(g) is its column mean.
5. **Universality against chance (element-shuffle null).** This tests whether infants agree on which element carries which value, beyond what their value distributions alone would give.
   a. For each pair (a, b), draw a random permutation π of the element labels and compare a's element e with b's element π(e), using the same π for all epochs of that pair.
   b. Recompute U_s(e) for every element and U_s overall.
   c. Repeat 10,000 times. The p-value for each element and for the overall value is (1 + number of null values ≥ observed) / (1 + 10,000).

6. **Bands.** Repeat items 1–5 for the weighted edge weight and the binary edge adjacency in each frequency band.

Step 3 — Within-session self-similarity (Comparison B)

This step measures how similar an infant is to itself within a few minutes. It is the ceiling for every other similarity, and 1 minus it quantifies moment-to-moment (state) variability.

1. **Split.** For each infant a and session s, split the K epochs into the first half (k = 1, …, K/2) and the second half (k = K/2 + 1, …, K). Pair epoch k with epoch k + K/2.
2. **Self.** Element level: W_s(a, e) = J between the two halves. Recording level: W_s(a) = S between the two halves. Group values: W_s(e) and W_s = means over the 20 infants.
3. **Matched others.** Compare infant a's first half with each other infant b's second half, using the same pairing (k, k + K/2). Element level: O_s(a, e) = mean over the 19 infants b ≠ a; recording level: O_s(a), likewise. This makes self and others directly comparable: same number of epoch pairs, same temporal separation.
4. **Within-session identifiability.** Per infant: W_s(a) − O_s(a) at recording level and W_s(a, e) − O_s(a, e) at element level. Test against 0 with the infant bootstrap (Step 7).
5. **Sensitivity.** Repeat with an odd/even split (odd k vs even k, paired k with k + 1).

Step 4 — Between-session self-similarity and identifiability (Comparisons C and D)

This step builds one 20 × 20 identifiability matrix per feature. Its diagonal is the infant compared with itself across sessions (C); its off-diagonal is the infant compared with other infants across sessions (D).

1. **Matrix.** M[a, b] = S_{(a,1),(b,2)} with epoch pairing (k, k). Rows are ses-1 and columns are ses-2. M is not symmetric: M[a, b] ≠ M[b, a].
2. **Global identifiability** (Amico & Goñi, 2018):

```
I_{\mathrm{self}} = \frac{1}{20}\sum_{a} M[a,a], \qquad I_{\mathrm{others}} =
\frac{1}{380}\sum_{a \neq b} M[a,b], \qquad I_{\mathrm{diff}} =
I_{\mathrm{self}} - I_{\mathrm{others}}
```

3. **Per-infant values.** self(a) = M[a, a]; others(a) = mean of the 38 off-diagonal entries in row a and column a; idiff(a) = self(a) − others(a). These 20 values feed Steps 6 and 7.
4. **Identification accuracy** (Finn et al., 2015).

   a. ses-1 → ses-2: for each row a, the predicted identity is the column b with the largest M[a, b]; it is correct if b = a.
   b. ses-2 → ses-1: for each column b, the predicted identity is the row with the largest value; it is correct if it equals b.
   c. A tie for the maximum counts as incorrect. Report both accuracies and their mean, in %. Chance is 1/20 = 5%.
5. **Element level.** For each element e, M_e[a, b] = J_{(a,1),(b,2)}(e), and I_self(e), I_others(e), I_diff(e) as in item 2. No element-level accuracy.
6. **Bands.** Repeat items 1–5 for the weighted edge weight and the binary edge adjacency in each frequency band.
7. **Interpretation note.** Development between sessions affects C and D alike on average, so I_diff isolates what is specific to the infant. NORB00064 (interval 4 days) is effectively a test–retest pair; report its self(a) and idiff(a) separately as well.

Step 5 — Combining the two axes

The two axes are combined in two ways: a per-element plane (universality vs individuality) and a per-element variance partition (trait vs session vs state).

5a. Universality–individuality plane. For each feature, plot one point per element: x = U(e) = mean of U_1(e) and U_2(e) (Step 2); y = I_diff(e) (Step 4). Colour by region (nodes) or region pair (edges); use marker shape for within- vs between-region. Classify every element with the FDR-corrected tests of Step 7:

Class	U(e) vs element-shuffle null	I_diff(e) vs permutation null
Universal	Significant	Not significant
Individual	Not significant	Significant
Universal and individual	Significant	Significant
Neither	Not significant	Not significant

For U(e), "significant" means significant at both sessions. Report the count of each class per feature, and list the elements in each class.

5b. Variance partitioning (state vs trait), per feature and element, on the raw epoch-level values x_{a,s,k}(e): 20 infants × 2 sessions × K epochs observations.

1. **Transform.** Apply a rank-based inverse-normal transform per element, pooling all observations of that element; this handles skew and many zeros (e.g., betweenness). Skip binary adjacency (0/1 values).
2. **Model.** $y = \mu + u_a + v_{a,s} + \varepsilon$, where u_a is the infant effect, $v_{a,s}$ the session-within-infant effect, and ε the epoch residual:

```
import statsmodels.formula.api as smf
m = smf.mixedlm("y ~ 1", df, groups=df["infant"], re_formula="1",
                vc_formula={"session": "0 + C(session)"})
r = m.fit(reml=True)
var_infant  = float(r.cov_re.iloc[0, 0])   # trait
var_session = float(r.vcomp[0])            # session-specific, incl.
development
var_epoch   = r.scale                      # state + measurement noise
```

3. **Report** each variance as a proportion of the total: infant (trait), session, epoch (state).
4. **Age-adjusted version.** Refit with "y ~ age_c", where age_c is age in days centred on the sample mean, so that the shared developmental trend is removed from the session term.
5. **Caveats to report.** With 2 sessions per infant the session term is weakly estimated; record non-converged fits as NaN and report how many there were. The group-shared component is captured by U(e), not by this model.

Step 6 — Developmental analyses

Three questions: does shared organisation change between sessions, does self-similarity decay with the time between sessions, and does similarity between infants fall with their age difference.

1. **Change in universality (ses-1 → ses-2).** Per feature: $\Delta U = U_2 - U_1$ (recording level), with per-infant values $\Delta u(a) = u_2(a) - u_1(a)$. Element level: $\Delta U(e) = U_2(e) - U_1(e)$. Test with the infant bootstrap (Step 7) and apply FDR across elements. Caveat to report: ages are more spread at ses-2 than at ses-1, so give the mean $|\Delta age|$ over the 190 pairs at each session next to ΔU, and interpret ΔU together with item 3.
2. **Self-similarity vs inter-session interval.** Across the 20 infants, compute the Spearman correlation between self(a) and the interval in days, and between idiff(a) and the interval. Obtain p from 10,000 permutations of the interval vector. Repeat with age at ses-1 in place of the interval. Report NORB00064 (4 days) and NORB00069 (about 42 days) individually.

3. **Similarity vs age difference (Mantel test)**, per feature and per session:
   a. Matrix 1: S_{(a,s),(b,s)} (20 × 20, symmetric). Matrix 2: D[a, b] = |age_a − age_b| in days at that session.
   b. Statistic: Spearman correlation between the 190 upper-triangle entries of the two matrices.
   c. Null: permute the infant order of D (rows and columns together) 10,000 times; two-sided p.

Sibling pair (only if Step 0 confirms NORB00086 and NORB00087 as twins or siblings). With one pair, all results are descriptive ranks, not significance tests. Run every item for all 26 feature sets, at recording and element level.

1. **Rank among all pairs.** At each session, report the percentile rank of the sibling similarity S_{(86,s),(87,s)} among all 190 pairs.
2. **Rank among matched pairs.** Compare the sibling similarity with (i) the 36 other pairs that involve infant 86 or infant 87, which controls for each sibling's general similarity to others, and (ii) non-sibling pairs with |Δage| ⩽ 7 days at that session, which controls for the siblings' identical age. Report the sibling value, the median and range of each comparison set, and the sibling's rank in each.
3. **Cross-session cross-identification.** In each identifiability matrix M, report M[86, 87] and M[87, 86] next to M[86, 86] and M[87, 87], and the rank of the sibling within its row and column. State whether either sibling would be "identified" as the other.
4. **Shared elements.** List the elements whose sibling J exceeds the 95th percentile of all non-sibling pairs at both sessions. Report how many of them are "individual" elements in Step 5a.
5. **Same-day control.** If the siblings were recorded on the same date, list any other infant pairs recorded on a common date and give their similarity for comparison; this separates a sibling effect from shared recording conditions.
6. **Figure.** For each feature, the distribution of the 190 pair similarities at each session, with the sibling pair and the matched pairs marked.

Step 7 — Statistics

The 190 pairs are not independent (each infant is in 19 of them), so every inference resamples or permutes infants, never pairs.

7.1 Descriptive. Report every summary as mean ± SD and as median with a 95% bootstrap confidence interval (CI). State what the SD is taken over (190 pairs or 20 infants). Do not compute standard errors, t-tests or p-values that treat the 190 pairs as independent observations.

7.2 Infant-level bootstrap (B = 10,000 resamples, percentile CI = 2.5th and 97.5th percentiles).

1. **Pair-based statistics** (Comparisons A and D, ΔU, region means). In each resample, draw 20 infant indices d_1, …, d_20 with replacement. Use all position pairs p < q with d_p ≠ d_q (a duplicated infant is not compared with itself), and take the mean of the precomputed pair value for (d_p, d_q). No networks are recomputed.
2. **Infant-based statistics** (Comparisons B and C, per-infant values such as idiff(a), Δu(a)). Ordinary bootstrap of the 20 values.
3. **I_diff from the matrix M.** In each resample: I_self = mean of M[d_p, d_p] over positions p; I_others = mean of M[d_p, d_q] over positions p ≠ q with d_p ≠ d_q.
4. **Paired differences** (weighted vs binary, feature 1 vs feature 2, ses-2 vs ses-1, within- vs between-region, region g1 vs g2). Compute both quantities in the same resample and keep the difference. Two-sided bootstrap p = 2 × min(fraction of differences ⩽ 0, fraction ⩾ 0), with a floor of 1/B.
5. Do not bootstrap identification accuracy: duplicated infants make the argmax ill-defined. Use the permutation test in 7.3 instead.

```
rng = np.random.default_rng(SEED)
def pair_bootstrap(pair_val, B=10_000):      # pair_val: 20 x 20 array, NaN
on diagonal
    out = np.empty(B)
    for i in range(B):
        d = rng.integers(0, 20, 20)
        p, q = np.triu_indices(20, k=1)
        keep = d[p] != d[q]
        out[i] = np.nanmean(pair_val[d[p][keep], d[q][keep]])
    return np.percentile(out, [2.5, 97.5]), out
```

7.3 Permutation tests (10,000 permutations; p = (1 + number of null values ⩾ observed) / (1 + 10,000); one-sided for self > others, two-sided for correlations).

1. **Global identifiability**, per feature. Permute the columns of M (ses-2 labels) with a random permutation; recompute I_diff and both identification accuracies.

2. **Element-wise I_diff(e).** Apply the same column permutation to every M_e in each iteration, so spatial dependence between elements is preserved; this gives one p-value per element.
3. **Universality against chance**: the element-shuffle null in Step 2, item 5.
4. **Correlations**: the Mantel test and the interval correlations in Step 6.

7.4 Multiple comparisons. Benjamini–Hochberg FDR at q < 0.05 (`statsmodels.stats.multitest.multipletests(p, method="fdr_bh")`), applied separately (i) across elements within each feature and test, and (ii) across the 26 feature sets for each global test.

7.5 Effect sizes. Report I_diff and ΔU in Jaccard units, accuracy in %, and correlations as ρ, each with its CI.

Step 8 — Robustness checks

Each check reruns the main results — U_1, U_2, I_diff, identification accuracy and the class counts of Step 5a — with one change at a time. Everything else is kept as in Steps 1–7.

Check	What changes	Features
0/0 rule	0/0 set to 0 (current code) instead of excluded	All
Epoch pairing	Average J and S over all K × K epoch pairs instead of (k, k)	All
Number of epochs	K = 6 (first 6 epochs) vs K = 12	All
Twins	Exclude NORB00086 and NORB00087 if Step 0 flags them	All
Pattern only	Divide each epoch's vector by its sum before computing S, removing global level differences	All (recording level)
Reference	`raw.set_eeg_reference("average")` before computing correlations	All weighted and binary features

Check	What changes	Features
Volume conduction	Debiased wPLI (`mne_connectivity.spectral_connectivity_epochs`, method `"wpli2_debiased"`, per band) instead of |r|; set negative values to 0	Weighted edge weight only
Binarisation	Keep the m strongest edges in every network instead of the GCC rule, where m = median edge count under the GCC rule (Step 0)	All binary features
Electrode distance	Spearman ρ of U(e) and of I_diff(e) with inter-electrode distance; repeat the within- vs between-region comparison on the residuals of U(e) after linear regression on distance	Edge features

Outputs to return

Please return the tables as CSV files, the figures as vector PDFs, and the intermediate arrays as .npy files with a short README listing each array's shape and axis order.

Tables

Table	One row per	Columns
T1 Global summary	Feature set (26)	U_1, U_2, W_1, W_2, I_self, I_others, I_diff, accuracy (both directions and mean), ΔU, Mantel ρ per session; each with mean ± SD, median [95% CI] and p (raw and FDR)
T2 Element results	Feature × element	Label, region, hemispheric category, within/between region, distance, U_1(e), U_2(e), W(e), I_diff(e), p, q, class (Step 5a), fraction of 0/0 cases
T3 Variance partition	Feature × element	Infant, session and epoch proportions, without and with the age term; convergence flag
T4 Robustness	Check × feature set	The T1 main numbers under each Step 8 check
T5 Bookkeeping	Session	Epoch counts per infant, τ_c and density (min, median, max), twin-check result

If the sibling pair is confirmed, also return T6 (one row per feature set: all values from the sibling analysis in Step 6) and the sibling figure.

Figures

1. Identifiability matrix M per feature: rows ses-1, columns ses-2, infants in the same order, one colour scale per feature, colour bar shown.
2. Per feature: U, W, I_self and I_others with 95% CIs, side by side.
3. Universality–individuality plane per feature (Step 5a).
4. Scalp maps: nodes coloured by I_diff(e); edges drawn only where I_diff(e) is FDR-significant.
5. Development: ΔU per feature with CI; scatter of self(a) against inter-session interval, with NORB00064 and NORB00069 labelled.
6. Variance partition: stacked bars (infant, session, epoch) per feature, averaged over elements and by region.
7. The existing heatmaps for ses-1 and ses-2, rows sorted by |Δage| from smallest to largest, colour scale fixed at 0–1, midline category labelled "M".